



Desenvolvimento de Software

Prof. Vinicius Cardoso Garcia
vcg@cin.ufpe.br :: @vinicius3w :: assertlab.com
http://viniciusgarcia.me

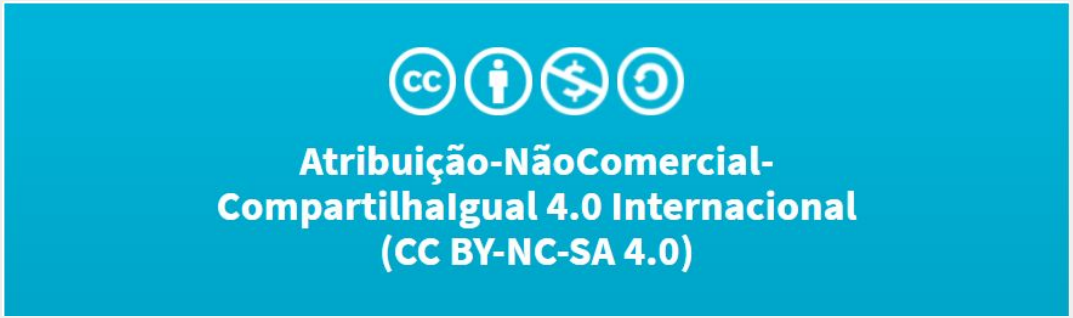




Universidade
Federal
de Pernambuco

Licença do material

Este Trabalho foi licenciado com uma Licença



Mais informações visite

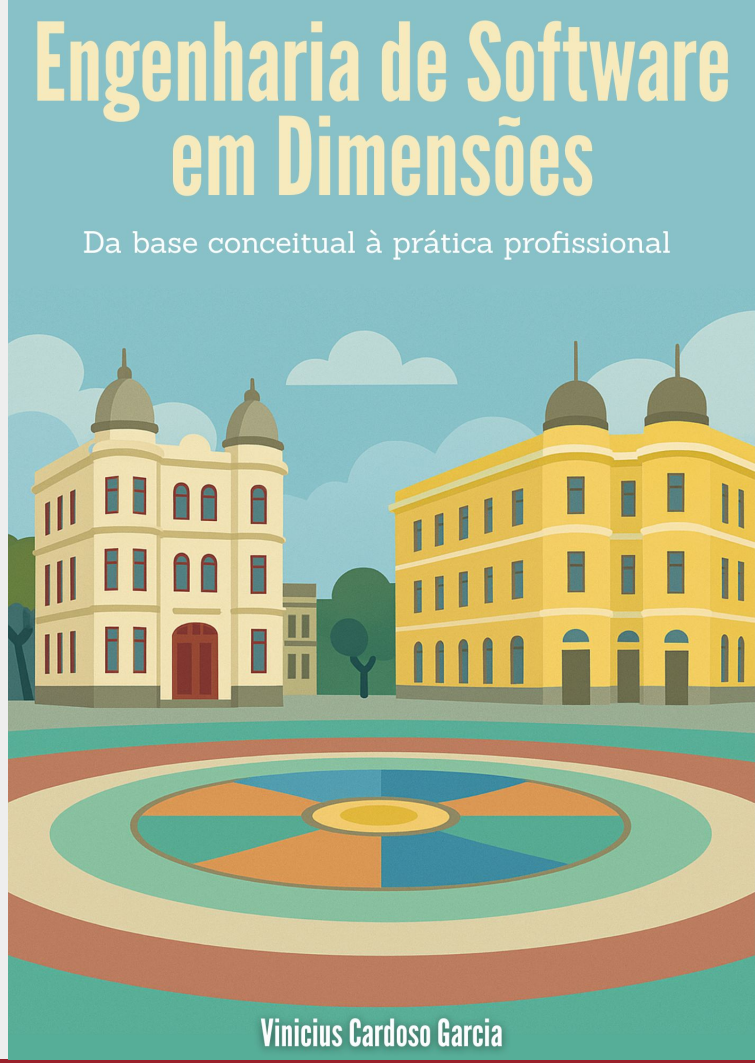
https://creativecommons.org/licenses/by-nc-sa/4.0/deed.pt_BR

Engenharia de Software em Dimensões

É um material abrangente, didático e atualizado que apresenta a Engenharia de Software como uma disciplina multidimensional. Cada capítulo foca em uma dimensão prática, conectando teoria e realidade profissional com clareza, profundidade e propósito.

Prof. Vinicius Cardoso Garcia

https://bit.ly/esdbook



Ponto de Commit

O podcast figital da disciplina (IF977) Engenharia de Software do curso de bacharelado em Sistemas de Informação do Centro de Informática (CIn) da Universidade Federal de Pernambuco (UFPE).

https://go.uaite.xyz/ponto-de-commit



Engenharia de Software Moderna

Engenharia de Software Moderna é um livro destinado a alunos de cursos de graduação em Computação. Ele pode ser lido também por profissionais que buscam um conhecimento horizontal da área.

Prof. Marco Tulio Valente

https://engsoftmoderna.info

@engsoftmoderna



Orquestrando Serviços com Docker Compose

Você já sabe subir um container. Agora é hora de fazer vários serviços conversarem entre si (frontend, backend e banco de dados) numa mesma máquina. Neste módulo, você vai dominar o Docker Compose do zero ao deploy e descobrir onde ele termina e o Kubernetes começa.



Agenda

01

O limite do Docker isolado

Por que rodar containers manualmente não escala para sistemas reais com múltiplos serviços.

02

Anatomia do docker-compose.yml

Cada diretiva explicada: services, build, ports, volumes, depends_on e muito mais.

03

Projeto PetFood completo

Um docker-compose.yml real para React + Express + PostgreSQL, comentado linha a linha.

04

Deploy e além do Compose

Padrão .env, Railway e Render por baixo dos panos, e onde o Kubernetes entra (e quando não entra).



O Limite do Docker Isolado

Rodar um único container funciona bem para experimentos. Mas aplicações reais são compostas por múltiplos serviços que precisam se comunicar, compartilhar redes e respeitar ordem de inicialização.

🐌 Processo manual

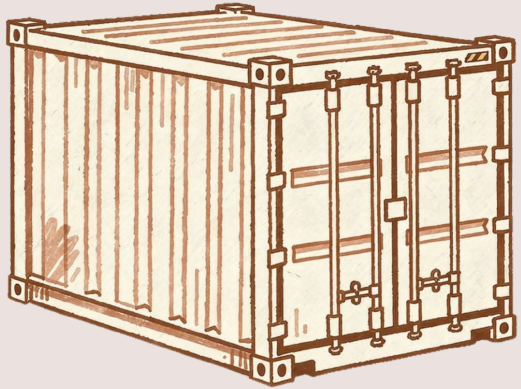
Para cada serviço: `docker build`, `docker network create`, `docker run` com dezenas de flags. Difícil de reproduzir e manter.

🔗 Dependências ignoradas

Nada garante que o banco suba antes do backend. Erros de conexão aparecem na hora menos esperada.

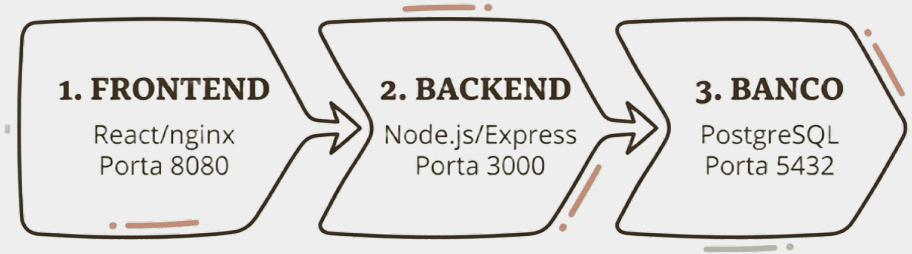
📋 Zero documentação executável

Um README com 20 comandos não é confiável. O docker-compose.yml *é* a documentação e o script ao mesmo tempo.



O PetFood não é um serviço — são três

O sistema PetFood é composto por três containers independentes que precisam se comunicar. Subir cada um manualmente exige múltiplos terminais, longos comandos docker run, criação manual de redes e passagem explícita de variáveis, para cada reinicialização.



⚠ Para subir os três com Docker puro: quantos terminais? Quantos comandos? Quantos parâmetros para memorizar?

Um arquivo. Um comando. Um ambiente.

O Docker Compose substitui uma sequência de comandos manuais por um único arquivo declarativo. Em vez de memorizar flags, redes e variáveis, você descreve o estado desejado, e o Compose faz acontecer.

❌ Sem Compose

```
docker network create petfood-net
docker run --network petfood-net \
  -e POSTGRES_DB=petfood postgres
docker run --network petfood-net \
  -e DATABASE_URL=... backend
docker run --network petfood-net \
  -p 8080:80 frontend
```

✅ Com Compose

```
docker compose up
```

Todos os serviços sobem, a rede é criada automaticamente, variáveis são injetadas, e a ordem de inicialização é respeitada.

O `docker-compose.yml` é o manual de montagem do seu ambiente. Quem lê esse arquivo sabe exatamente como o sistema funciona.



A estrutura do arquivo

O `docker-compose.yml` segue uma estrutura hierárquica clara. Cada serviço declara como seu container deve ser construído, configurado e conectado aos demais. O Compose cria automaticamente uma rede interna — todos os serviços se enxergam pelo nome, sem configuração adicional.

```
services:               # os containers que compõem o sistema
  nome-servico:
    image: ...          # imagem pronta do Docker Hub
    build: ...          # ou: Dockerfile local
    ports: ...          # mapeamento host:container
    environment:        # variáveis de ambiente
      CHAVE: valor
    depends_on:         # ordem de inicialização
      - outro-servico
    volumes:            # persistência de dados
      - nome:/caminho

volumes:                # volumes nomeados (dados que sobrevivem ao down)
```

ⓘ O Compose cria uma rede interna automaticamente, todos os serviços se enxergam pelo nome do serviço, não por IP.



Diretiva: `services`

O bloco `services` é o coração do arquivo. Cada chave dentro dele define um serviço — que se tornará um container em execução.

```yaml
services:
  frontend:   # nome do serviço (também vira hostname na rede)
    build: ./frontend
    ports:
      - "80:80"

  backend:
    build: ./backend
    ports:
      - "3000:3000"

  db:
    image: postgres:15
```

O nome do serviço (`frontend`, `backend`, db) é usado como hostname DNS interno. Outros containers podem se conectar usando exatamente esse nome.



image vs build

image: usar uma imagem pronta

```
services:
  db:
    image: postgres:15

  cache:
    image: redis:7-alpine
```

Use quando a imagem já existe no Docker Hub ou em um registry privado. Ideal para serviços de infraestrutura como bancos de dados e caches.

build: construir localmente

```
services:
  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile

  frontend:
    build: ./frontend
    # shorthand: context=./frontend
```

Use quando você tem o código-fonte e precisa gerar a imagem. O Compose chama docker build automaticamente antes de subir o container.



`ports` vs `expose`

`ports`: publicar para o host

```
"HOST:CONTAINER"
- "80:80"
- "3000:3000"
```

Mapeia uma porta do seu computador para o container. Qualquer processo no host (e na internet, se houver firewall aberto) pode acessar.

`expose`: apenas entre serviços

```
expose:
  - "5432"
```

Não publica nada no host: só documenta a porta interna. O que protege o banco é **não** usar `ports`.

⚠ Nunca publique a porta do PostgreSQL (`5432`) com `ports` em produção. Sem `ports`, ela fica restrita à rede interna do Compose.



`environment` — Variáveis de Ambiente

Configura variáveis de ambiente dentro do container. Existem duas formas equivalentes:

Forma de lista

```
environment:
  - POSTGRES_USER=petfood
  - POSTGRES_PASSWORD=secret
  - POSTGRES_DB=petfooddb
```

Forma de mapa (recomendada)

```
environment:
  POSTGRES_USER: petfood
  POSTGRES_PASSWORD: secret
  POSTGRES_DB: petfooddb
```

Para não deixar senhas no arquivo versionado, use referências ao arquivo .env:

```
environment:
 POSTGRES_PASSWORD: ${DB_PASSWORD} # lido do .env automaticamente
```



depends_on — Ordem de Inicialização

Define que um serviço só deve iniciar *depois* de outro. Evita que o backend tente conectar ao banco antes de ele estar pronto.



```yaml
services:
  backend:
    build: ./backend
    depends_on:
      db:
        condition: service_healthy  # aguarda o healthcheck passar

  db:
    image: postgres:15
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U petfood"]
      interval: 5s
      timeout: 5s
      retries: 5
```

`service_started` aguarda o container subir. `service_healthy` é mais robusto e aguarda o healthcheck passar, preferível para bancos de dados.

`volumes` — Persistência de Dados

Volume nomeado (produção)

```yaml
services:
  db:
    image: postgres:15
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:   # declarado aqui embaixo
```

Gerenciado pelo Docker. Os dados sobrevivem ao `docker compose down`. Ideal para bancos em produção.

Bind mount (desenvolvimento)

```yaml
services:
  backend:
    build: ./backend
    volumes:
      - ./backend:/app
      - /app/node_modules
```

Mapeia uma pasta do seu computador diretamente. Alterações no código são refletidas em tempo real dentro do container, perfeito para hot reload no desenvolvimento.



O DNS Interno do Compose

Um dos superpoderes menos conhecidos do Compose: **o nome do serviço vira o hostname** na rede interna. Nenhuma configuração extra é necessária.



Do backend ao banco

Use o nome do serviço, não `localhost` (no Compose, ele aponta para o próprio container):

`postgresql://db:5432/petfooddb`



Rede criada automaticamente

O Compose cria uma rede bridge privada para todos os serviços do arquivo. Sem `docker network create` manual.



Isolamento por padrão

Serviços de projetos diferentes ficam isolados em redes separadas. Sem colisão de nomes entre projetos distintos.



Apresentando o Sistema PetFood

O PetFood é um e-commerce de ração para pets com arquitetura de três camadas: interface em React servida pelo nginx, API REST em Node.js/Express e banco de dados PostgreSQL. É o projeto que usaremos para montar um docker-compose.yml completo e comentado.

Frontend React + Vite buildado e servido pelo nginx na porta 80	**Backend** Node.js + Express, API REST na porta 3000
Database PostgreSQL 15 com volume persistente	



docker-compose.yml — Parte 1: Cabeçalho e Frontend

```yaml
# PetFood — docker-compose.yml
# Sobe frontend (React/nginx), backend (Express) e banco (PostgreSQL)
services:
  # ── FRONTEND ──────────────────────────────────────────
  frontend:
    build:
      context: ./frontend       # pasta com o Dockerfile do React
      args:
        VITE_API_URL: /api      # o Vite embute no build; o nginx faz o proxy
    ports:
      - "8080:80"               # acessível em http://localhost:8080
    depends_on:
      - backend                 # sobe somente após o backend iniciar
```

O nginx dentro do container do frontend serve os arquivos estáticos buildados e faz proxy das chamadas `/api/*` para o serviço `backend` usando o DNS interno do Compose.

Sistema completo em uma linha de fluxo:
`http://localhost:8080` → frontend → `http://backend:3000` → banco

docker-compose.yml — Parte 2: Backend

```yaml
# ─── BACKEND ──────────────────────────────────────────────
backend:
  build:
    context: ./backend        # pasta com o Dockerfile do Express
  ports:
    - "3000:3000"             # expõe a API para o host (útil no dev)
  depends_on:
    db:
      condition: service_healthy  # aguarda o postgres estar pronto
  environment:
    NODE_ENV: production
    DATABASE_URL: postgresql://${DB_USER}:${DB_PASSWORD}@db:5432/${DB_NAME}
    # 'db' é o hostname resolvido pelo DNS interno do Compose ^^^
    PORT: 3000
  env_file:
    - .env                    # carrega variáveis do arquivo .env local
```

Perceba que a DATABASE_URL aponta para `db`, o nome exato do serviço PostgreSQL definido abaixo. Isso é o DNS interno do Compose em ação.



docker-compose.yml — Parte 3: Banco de Dados

```
  # ─── DATABASE ──────────────────────────────
  db:
    image: postgres:15-alpine   # imagem oficial, versão leve
    expose:
      - "5432"                  # visível só na rede interna
    environment:
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: ${DB_NAME}
    volumes:
      - pgdata:/var/lib/postgresql/data       # dados persistentes
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER}"]
      interval: 5s
      timeout: 5s
      retries: 5

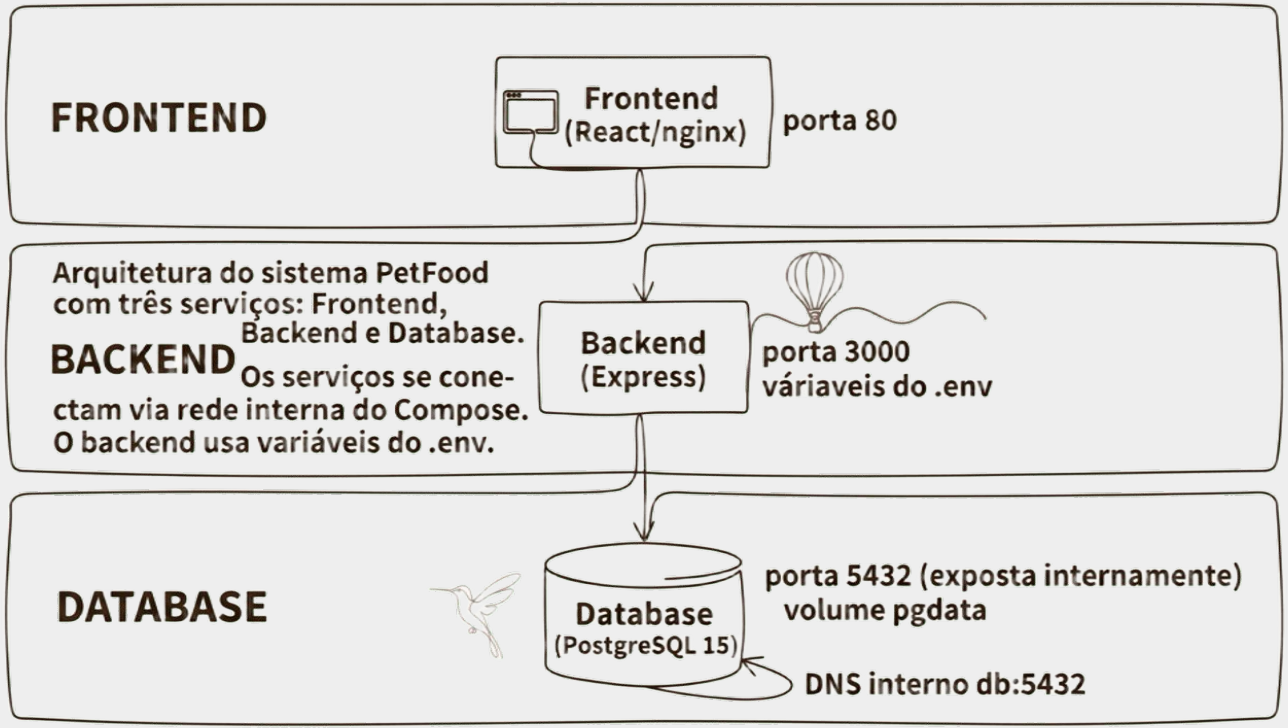
# ─── VOLUMES NOMEADOS ──────────────────────────
volumes:
  pgdata:   # declaração obrigatória para volumes nomeados
```

ℹ️ 🗄️ **expose vs ports:** `expose` abre a porta apenas para a rede interna do Compose. O banco não fica exposto para o mundo, só o backend o acessa.

⚠️ 💾 Sem o volume `pgdata`, todos os dados são perdidos a cada `docker compose down`. O volume garante persistência entre reinicializações.



O Arquivo Completo — Visão Geral



O Compose gerencia toda essa complexidade com um único arquivo declarativo. A ordem de boot, a rede interna, os volumes e as variáveis de ambiente são todos orquestrados automaticamente.

Subindo e Gerenciando os Serviços

`docker compose up -d`

Sobe todos os serviços em background (detached). Use `--build` para forçar rebuild das imagens antes de subir.

`docker compose logs -f backend`

Acompanha os logs de um serviço específico em tempo real. Omita o nome para ver todos os serviços juntos.

`docker compose ps`

Lista o status de todos os containers do projeto: nome, estado, portas mapeadas.

`docker compose exec backend sh`

Abre um shell interativo dentro de um container em execução. Útil para debug e inspeção.



Stop vs Down vs Down -v

Três comandos que parecem iguais mas têm efeitos muito diferentes. Saber a diferença evita dores de cabeça, especialmente em produção.

`docker compose stop`

Para os containers sem removê-los. Dados e estado são preservados. Use quando quiser pausar temporariamente e retomar com `docker compose start`.

`docker compose down`

Para e remove os containers e as redes criadas. Os volumes nomeados são mantidos, seus dados do banco continuam intactos.

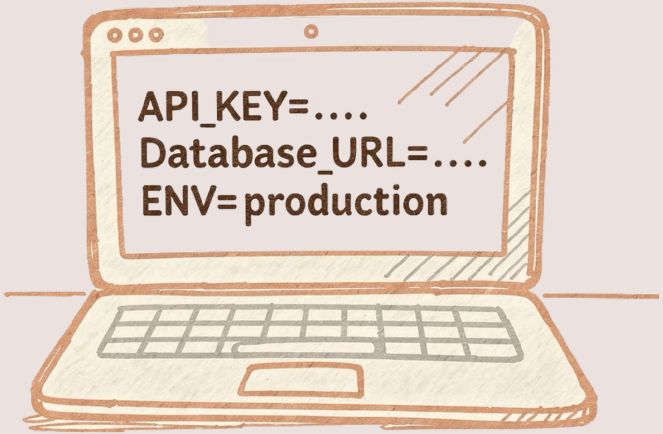
`docker compose down -v`

Remove tudo, incluindo os volumes nomeados. O banco de dados é apagado permanentemente. Use com cautela, ideal para resetar o ambiente de dev.

⊗ Nunca execute `docker compose down -v` em produção sem backup. Este comando é destrutivo e irreversível para os dados persistidos em volumes.



Variáveis de Ambiente: Padrão .env + .env.example



Senhas e segredos não devem jamais ser commitados no Git.
O padrão da indústria é usar dois arquivos complementares.

.env (nunca commitar)

```
DB_USER=petfood
DB_PASSWORD=minha_senha_secr
eta
DB_NAME=petfooddb
JWT_SECRET=abc123supersecret
```

Adicione ao `.gitignore` imediatamente. Contém os valores reais usados localmente.

.env.example (sempre commitar)

```
DB_USER=
DB_PASSWORD=
DB_NAME=
JWT_SECRET=
```

Template público com as chaves necessárias, mas sem valores sensíveis. Novos membros do time copiam e preenchem com seus próprios valores.

Usando Variáveis no `docker-compose.yml`

O Compose carrega o arquivo `.env` da pasta atual automaticamente. Você pode referenciar qualquer variável com a sintaxe `${VARIAVEL}`.

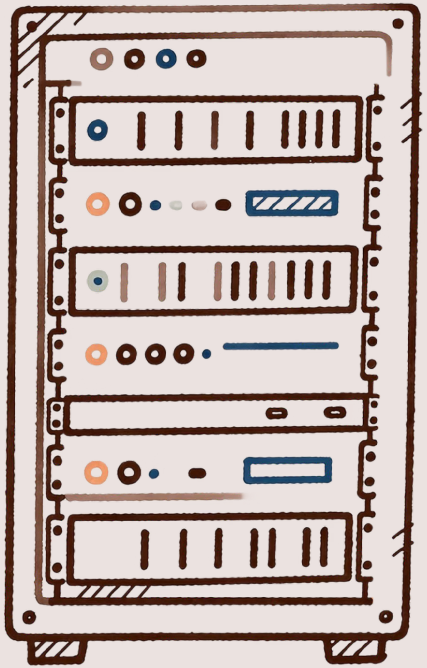
```
# .env
DB_USER=petfood
DB_PASSWORD=s3cr3t
DB_NAME=petfooddb
BACKEND_PORT=3000
```

```
# docker-compose.yml
services:
  backend:
    ports:
      - "${BACKEND_PORT}:3000"   # porta configurável pelo .env
    environment:
      DATABASE_URL: postgresql://${DB_USER}:${DB_PASSWORD}@db:5432/${DB_NAME}

  db:
    image: postgres:15
    environment:
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: ${DB_NAME}
```

O mesmo `docker-compose.yml` roda em qualquer máquina de dev ou teste, mudando só o `.env`. Produção é outra história.





Conectando com Plataformas de Deploy

Plataformas como **Railway** e **Render** simplificam o deploy de aplicações containerizadas, mas é importante entender o que acontece por baixo dos panos para depurar problemas e tomar boas decisões de arquitetura.

Railway — Como Funciona por Baixo

1

Push no Git

Você faz push para o repositório conectado ao Railway.

2

Build automático

Railway detecta o Dockerfile e executa o build. Sem docker-compose.yml em produção, cada serviço é deployado separadamente.

3

Serviços privados

O banco e outros serviços internos ficam em rede privada. Railway injeta a variável `DATABASE_URL` automaticamente.

4

DNS interno

Serviços se comunicam pelo hostname gerado pela plataforma, o mesmo conceito do DNS interno do Compose, mas gerenciado pelo Railway.

O `docker-compose.yml` é a ferramenta de desenvolvimento. Em produção no Railway, você configura cada serviço individualmente pela dashboard ou pelo arquivo `railway.json`.



Render — O que Acontece por Baixo

Web Services

O Render detecta seu `Dockerfile` e faz o build. Você define as variáveis de ambiente pela dashboard, o equivalente ao seu `.env` local.

Banco de Dados

O Render oferece PostgreSQL gerenciado. Ele gera uma `DATABASE_URL` privada que você cola como variável de ambiente do seu backend, idêntico ao que você faz no Compose com o DNS interno.

O paralelo com o Compose

Compose: `depends_on`

Render: deploy em ordem manual ou via health checks da plataforma.

Compose: rede interna

Render: rede privada com hostname gerado automaticamente.

Compose: volumes nomeados

Render: discos persistentes configurados por serviço.





E o Kubernetes?

Railway e Render escondem a orquestração em produção. Mas quando o mercado fala em **orquestrador**, quase sempre está falando do **Kubernetes**. O que ele faz que o Compose não faz, e quando vale o custo?

Três degraus, três perguntas

Docker, Compose e Kubernetes não competem. Cada degrau responde a uma pergunta que o anterior não responde.

`Docker`

Como empacoto minha aplicação para rodar igual em qualquer lugar?

1 container · `Dockerfile`

`Docker Compose`

Como subo juntos os vários serviços do meu sistema?

1 máquina · `docker-compose.yml`

`Kubernetes`

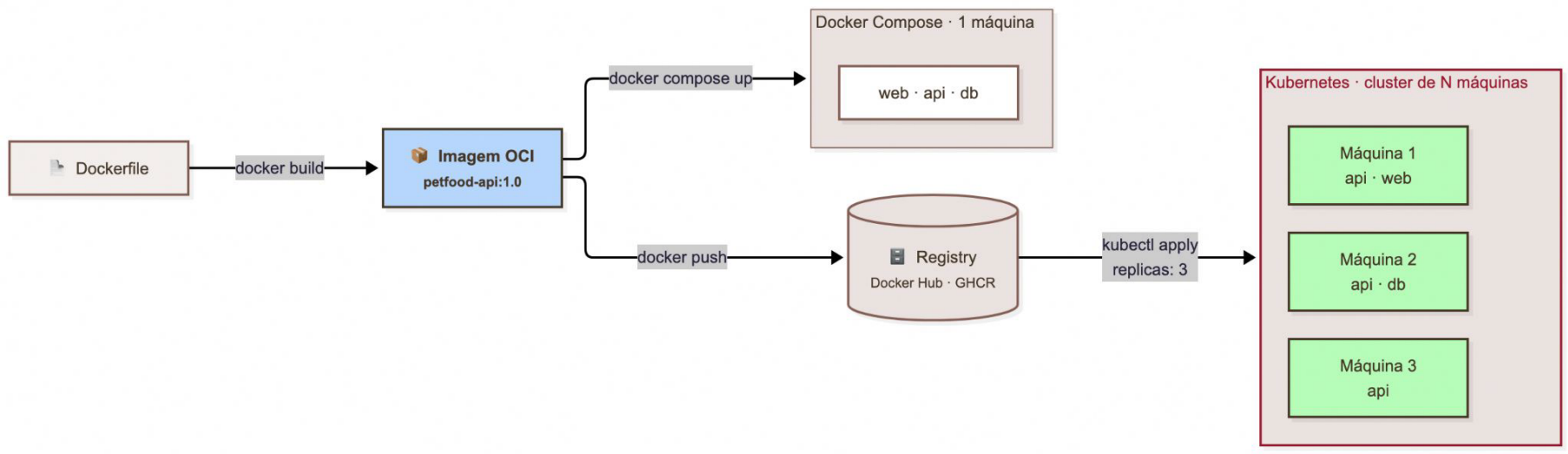
Como mantenho o sistema no ar em várias máquinas, mesmo com falhas e picos de carga?

N máquinas · `manifestos YAML`

A imagem é o elo comum: o `Dockerfile` da aula passada gera exatamente o que um cluster Kubernetes executaria. Nada do que vocês aprenderam é descartado.



Da imagem ao cluster



O Kubernetes não constrói imagens: ele executa as mesmas imagens OCI que o `docker build` gera. Desde a versão 1.24 ele trocou o motor interno, não o formato das imagens.

Declarar uma vez × vigiar sempre

`compose up`: aplica uma vez

```
services:
  api:
    image: petfood-api:1.0
```

Sobe o estado descrito numa única máquina. Se ela cair, o sistema cai junto. Escalar é manual.

`replicas: 3`: vigia sempre

```
kind: Deployment
spec:
  replicas: 3   # sempre 3 cópias
```

Compara o tempo todo o que está rodando com o que foi declarado. Se um container cai, sobe outro; se uma máquina falha, as cópias vão para outra.

⚠ Analogia: com Docker você diz “asse um bolo”. Com Kubernetes você declara “quero sempre 3 bolos na vitrine”, e o gerente da confeitaria cuida do resto.



O que o Compose já te ensinou

1

`Nome do serviço` → Service

O DNS pelo nome continua valendo: `db:5432` na `DATABASE_URL` faria o mesmo sentido num cluster.

2

`.env` → Secret e ConfigMap

Credencial nunca no código nem no repositório. Volume nomeado vira PersistentVolumeClaim.

3

`healthcheck` → probes

Readiness e liveness probes: só mandar tráfego ao container pronto e reiniciar o que travou.

4

`depends_on` → não existe

Não há ordem de boot garantida: a aplicação precisa tolerar o banco fora do ar e tentar de novo.



Quando o Kubernetes vale o custo?

1

Mais de uma máquina

O volume de uso ou a exigência de disponibilidade não cabem num único servidor.

2

Carga que varia muito

A escala precisa acompanhar os picos sem ninguém intervir manualmente.

3

Indisponibilidade cara

Pagamentos, saúde, serviço público: cada minuto fora do ar tem custo real.

4

Muitas equipes, muitos serviços

E gente para operar o cluster, ou orçamento para um serviço gerenciado (EKS, GKE, AKS).

Para o projeto de vocês, Compose numa máquina ou Render/Railway resolvem. Kubernetes é uma ótima resposta para um problema que vocês ainda não têm: escolha a ferramenta pelo problema, não pela fama.

Checklist de Boas Práticas

1

Versione o docker-compose.yml

O arquivo deve estar no repositório. É a documentação executável do projeto.

2

Nunca commite o .env

Sempre adicione `.env` ao `.gitignore` e mantenha o `.env.example` atualizado.

3

Use healthchecks reais

Prefira `condition: service_healthy` no `depends_on` para garantir que o banco esteja pronto de fato.

4

Não exponha o banco para o host

Use `expose` em vez de `ports` para o PostgreSQL em ambientes que não sejam exclusivamente de desenvolvimento.



Recapitulando — O que Aprendemos

Compose resolve o caos

Um arquivo substitui dezenas de comandos manuais e garante reprodutibilidade total do ambiente.

DNS interno é poderoso

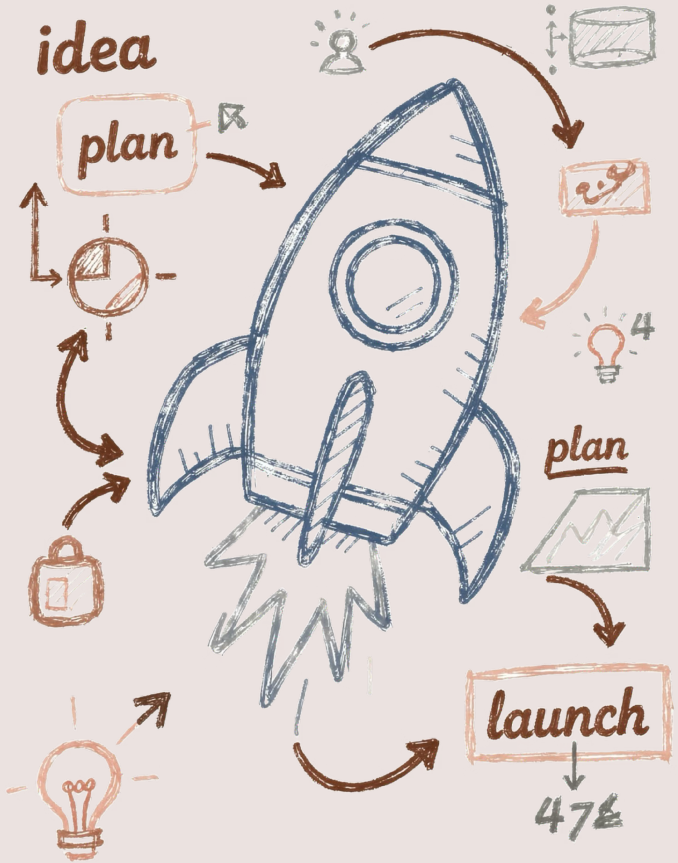
O nome do serviço é o hostname. Use-o nas connection strings em vez de IPs ou localhost.

Stop ≠ Down ≠ Down -v

Cada comando tem um impacto diferente nos dados. Conheça a diferença antes de usar em produção.

Plataformas e orquestradores

Railway, Render e Kubernetes levam os princípios do Compose à produção. Kubernetes só quando o problema pede.



Próximos Passos

Pratique agora

Clone o repositório do seu projeto, rode `docker compose up --build` e explore os logs de cada serviço.

Experimente o down -v

Suba o sistema, insira dados no banco, rode `docker compose down -v` e suba novamente. Observe o que acontece.

Faça o deploy

Conecte o repositório ao Railway ou Render, configure as variáveis de ambiente e veja seu sistema de três camadas rodando na nuvem.

Você agora sabe compor sistemas reais com Docker Compose e sabe onde o Kubernetes entra: quando o problema pede várias máquinas, escala automática e alta disponibilidade. 🚀



contacts

vcg@cin.ufpe.br
@vinicius3w
viniciusgarcia.me

academy

go.uaite.xyz/vcg-es
go.uaite.xyz/vcg-microservices
go.uaite.xyz/vcg-devops
go.uaite.xyz/vcg-esaia
go.uaite.xyz/vcg-deso

Centro de Informática UFPE
Universidade Federal de Pernambuco
ASSERT LAB

Desenvolvimento de Software (CIN0136)

Orquestrando Serviços com Docker Compose